Shared Libraries in Windows and Linux

Prise de notes sur les librairies partagées dans Windows et Linux a partir de la conférance de Ofek Shilon.

Introduction

At the outset of his presentation, Ofek Shilon explains that he primarily worked on Windows throughout his career but transitioned to Linux about four years ago. The transition was challenging, particularly concerning the linker. In this presentation, he will delve into the differences between Windows and Linux regarding shared libraries, with a focus on the linker and loader.

Slide [1, 2, 3] - Introduction to the Linker

Slide 1

Step 1: Compilation

- Source -> Object Files
- Object Files = containers for sections (code and data)

Step 2: Linking

- Gathers identically named sections from different object files
- Concatenates them into a large section in the final binary
- Reorders sections to make them adjacent on disk if they require the same runtime permissions (forming segments)

Step 3: Loading

- Upon program startup, the loader maps segments to separate memory locations
- Adjusts page permissions accordingly

Slide 2

Code Sections:

- Typically contain numerous function calls
- In the simplest case, calls are internal to the binary itself
- Generally, the process also loads shared libraries into memory

Using Shared Libraries:

- An executable can call a function implemented in a shared library
- This shared library can itself call a function in another shared library
- The primary role of the loader and linker is to properly connect these calls

Terminology:

- On Linux, shared libraries are called "Shared Objects" (SO(s))
- On Windows, they are called "Dynamic Link Libraries" (DLL(s))
- The author will use the general term "shared library" for both
- He will also use the term "binary" to refer to both an executable and a shared library
- Most of what is said applies to both.

Slide 3

Naive View of Function Call Wiring

Code Example:

- If your code includes a function "foo" that calls "bar"
- "bar" is not explicitly written in the binary's code section itself

Encoding in the Binary:

- The call to "bar" is encoded by a "call" instruction with a null address (placeholder)
- The binary also contains a section generated by the linker called "relock" for relocations
- Relocations are small instructions to wire calls of the form "find the function bar and replace the placeholder with the found address"

Slide 4 - Import Data Section

"Imports" refers to the libraries that the executable announces it needs to import, i.e., those it depends on. The "idata" section contains this information about imports.

Slide 5 - Main Structure of the "idata" Section

Directory Table:

- Within the "idata" section, the main structure is called the "directory table."
- This table contains "directory entries," each pair corresponding to a DLL.

Contents of Directory Entries:

- Each directory entry contains the name of the DLL and a reference to an "import address table."
- The import address table is essentially a list of names or symbol indices to import from that DLL.

Symbol Resolution:

- The loader uses the data from the import lookup table to resolve symbols.
- It then places their addresses into the "import address table."
- There is only one import address table, but each directory entry contains an offset to it.

Slide 6 - Import Address Table - Creating and Using DLLs on Windows

Function Export:

- When creating a DLL on Windows and exporting a function, you use the `__declspec(dllexport)` directive to mark the function.
- This makes the function available to other binaries linking against your DLL.

Exports Section:

- When a function is exported, an entry is created in the E4 (exports) section of the DLL.
- This entry is an "export stub" that jumps to the actual implementation of the function.

Creation of Import Binary:

- When creating a DLL on Windows, another binary is also generated, called the "static import file."
- This file is static, meaning it's intended to be merged into the final executable.

Merge into Executable:

- The linker merges the static import file into the final executable.
- This adds to the final binary an "import stub" that simply jumps to the "export stub" in the DLL.
- It also adds a directory entry in the "idata" section for this DLL.

Loader Loading:

- Upon loading, the loader uses the entry in the "idata" section to locate the DLL and link it to the executable.
- It finds the appropriate addresses and writes them into the "import address table."

Call Resolution:

- When a function is called from the executable, the call is resolved by the linker by first jumping to the import table.
- From there, it jumps to the DLL's export table, and then to the actual implementation of the function.

Slide 7 - Semantic Interface on Windows

General Principle:

- On Windows, each binary exposes an interface per library.
- A Windows binary indicates to the outside world that it needs to find a certain library and use the symbols defined within it.
- Each library is identified and used distinctly.

Slide 8 - Import Sections on Linux

Presence of Two Import Sections:

- Unlike Windows, a Linux binary contains two distinct import sections.
- The first section, called the "dynamic section" (.dynamic), is simply a list of library names.
- The second section, called the "symbol table" (.dynsim), contains specific symbols from these libraries.

Schematic Description:

- Schematically, the first section lists the names of libraries the binary needs.
- The second section lists the specific symbols needed from these libraries.

Slide 9 - Semantic Interface on Linux

Announcement of Required Libraries:

- A Linux binary separately announces to the outside world the libraries it needs.
- It also specifies which symbols it needs to use from each of these libraries.
- This communication isn't limited to executables but also applies to shared libraries.

Symbol Search Order:

- When resolving symbols for a shared library, by default, the program binary is first searched for these symbols.
- This is done before even looking into dependent libraries and even before the library itself.

Behavior Control:

- This search behavior can be controlled using certain compilation switches.

Slide 10 - Conclusion on Semantic Differences

Under Windows:

- A Windows binary explicitly indicates the libraries it wants to import and the specific symbols it wants to use from each library.

Under Linux:

- A Linux binary announces the libraries it needs and indicates which symbols should be searched within these libraries, without specifying their exact location.

Implications:

- This Linux approach opens up interesting possibilities but can lead to conflicts if two shared libraries implement the same symbol.
- However, it also allows for some flexibility, such as substituting a third-party function with a custom implementation.

Slide 11 - Introduction to Position Independent Code

Basic Forms of Calls:

- The simplest form of a call is a call to a fixed address, which is not "position independent."
- Another form is a call with a relative offset from the current program counter, which is "position independent."

Using the Global Offset Table (GOT):

- The primary form of position independent code involves making a call to a fixed offset in the Global Offset Table (GOT).
- The GOT is a section generated at load time by the loader and contains a list of resolved function addresses.
- The call is made to the address stored in this location in the GOT, ensuring the call is "position independent."

Understanding Positioning:

- It's important to note that while the entire binary might not be position independent, the code section is.
- Position independent code was a major goal in the design of the Linux system, especially for shared libraries.
- This allows for sharing code pages among processes using the same libraries, thus saving memory.

Internal Binary Calls:

- This call flow also applies to internal calls within the binary, not just imported or exported functions.

Slide 12 - Discussion on Position Independent Code in Linux

- All code in shared libraries under Linux is position independent.
- To compile position independent code, a switch needs to be specified.
- If an object file hasn't been compiled with the appropriate switch, a linking error occurs, prompting to recompile with the "-fpic" option.
- There are two similar switches for position independence (-fpic and -fPIC), but they differ slightly on some architectures.
- ELF executables were historically loaded at the address they were built for, but the introduction of ASLR allowed them to be loaded at any address.

Slide 13 - Resolution Time

- In Linux executables, symbol resolution is checked at link time.
- A Linux executable will fail to link if certain specified symbols are not found in the specified libraries.

- However, for dynamic libraries, Linux can link them without searching for these missing symbols.
- The switch that controls this behavior is "--no-allow-shlib-undefined."
- There are other more nuanced switches to intervene in this behavior, such as "--no-allow-sleep-undefined", but their functionality is not detailed here.

Slide 14 - Intermediate Summary on Linux

- Dependencies in terms of symbols and libraries are listed separately on Linux.
- By default, all calls are indirect and go through the Global Offset Table (GOT).
- By default, symbol resolution in libraries is performed at load time, with the executable being searched first.
- These facts have significant implications for developers, some of which will be addressed later in the presentation.

Slide 15 - How to Create a Process-Scale Singleton?

- *A singleton is a class or data structure that can only be instantiated once.*
- Under Linux, to form a process-scale singleton, simply place the implementation within the executable itself.
- Wherever the singleton is used, its implementation is retrieved from the executable, where the definition is searched for first.
- Under Windows, the task is more complex: for a process-scale singleton, planning ahead is required, and all binaries need to be linked to a single DLL implementing the singleton.

Slide 16 - Can There Be Circular Dependencies Between Libraries?

- *A circular dependency between libraries means that library A depends on library B, and library B depends on library A (or any other circular cycle).*
- Under Linux, it's possible to have circular dependencies between libraries without needing any special actions.
- By default, shared libraries are built with delayed symbol resolution, allowing all implementations in all libraries to be available to the loader.
- Conversely, under Windows, circular dependencies between libraries are not allowed by design.

Author's Note: The presenter expresses a preference for the Windows approach, believing it encourages better organization of shared libraries. This opinion is shared by other experts, such as Rui Ueyama, the author of "LLD and MOLD linkers," who suggests that using circular dependencies under Linux is a bad practice.

Slide 17 - Replacing a Symbol from a Shared Library from an Executable

- Under Windows, it's not possible to replace a symbol from a shared library from an executable.
  - Dependencies are strictly defined, and there's no room to choose which symbol to take from which library.
- Under Linux, it's possible to replace a symbol from a shared library from an executable.
  - This can be achieved by declaring the symbol in the executable, which will override other implementations of that symbol unless exotic compilation switches are used.
- A classic example of symbol interposition is custom implementation of memory management functions (such as new and delete) in a C++ program.
- On Windows, definitions of these functions in the program cannot be used instead of default versions provided by the implementation, which is not compliant with the C++ standard.

Slide 18 - Lazy Binding on Windows and Linux

- Under Windows, lazy binding is called "delayed loading," which defers the resolution of function addresses until their first use.
  - This helps avoid resolving thousands of unused functions in a large library, which can substantially reduce loading time.
  - By default, lazy binding is not enabled on Windows but can be added with a linker switch to specify a DLL to be loaded lazily.
- Under Linux, lazy binding is the norm and is enabled by default.
  - This means that the resolution of function addresses is deferred until they are actually called.
  - Intervention in this behavior can be achieved with the LD_BIND_NOW environment variable, which modifies the behavior at the system level.

Slide 19 - Procedure Linkage Table (PLT)

- The PLT is an executable section generated by the loader under Linux, which plays a crucial role in resolving function calls.
- Initially, a call is made to a fixed offset in the PLT, where the first instruction is a jump to a fixed location in the Global Offset Table (GOT).
- The GOT initially contains the address of a place that performs the actual resolution of the function.
- The loader, always loaded into the process, performs this resolution upon the first call to the function and updates the address in the GOT with the actual function address.
- From the second call onwards, the call is made directly to the actual implementation of the function, passing through the PLT and the GOT.

Slide 20 - Function Pointer Comparison (Linux)

C++ Standard: *If two function pointers are null or point to the same function, they are considered equal.*

- Under Linux, function pointer comparison is facilitated by a significant section called ".symtab" (symbol table).
- However, actual calls are made to addresses in different PLTs for executables and shared libraries.
- To enable function pointer comparison, the loader populates the symbol table with the PLT address for each function during loading.
- This allows for comparing function pointers, even if they point to different addresses in the PLT.

Slide 21 - Function Pointer Comparison (Windows)

- Under Windows, with DLLs loaded lazily, the address of a function before its call differs from its address after its call.
- This means that two function addresses pointing to the same function can be considered different before and after the call.
- Therefore, under Windows, a function pointer is not equal to itself in the same binary, let alone to a function pointer in different binaries.
- In the presence of delayed loading, Windows does not adhere to the function pointer comparison standard as defined by the C++ standard.

Slide 22 - Symbol Visibility (Windows)

- Under Windows, symbol visibility is controlled by various compilation directives.
- "__declspec(export)" can be used to mark symbols for export, which adds them to the "idata" section of the executable.
- "__declspec(import)" can also be used to mark symbols for import, but this does not add them to the "idata" section.
- By default, a symbol is neither exported nor imported and is considered internal to the binary.

Slide 23 - Symbol Visibility (Linux)

- In Linux, there are no analogous import and export directives.
- The distinction is made between public and hidden symbols.
- Public symbols go through the Procedure Linkage Table (PLT) and the Global Offset Table (GOT) and are visible to other libraries and executables.
- By default, symbol visibility is public, meaning they are both imported and exported.
- Visibility can be modified with directives like "fvisibility=hidden" to make them hidden.
- This feature can significantly improve linking and loading times of shared libraries and is highly recommended.
- Hidden symbols were added late in the design of ELF and are documented by the GCC (GNU Compiler Collection, the standard C++ compiler on Linux) manual.

Slide 24 - Loader

- The loader is an entity that can have multiple names such as "Dynamic Linker," "Interpreter," "Image Loader," etc.
- Under Linux, it's a file called "ld-linux-x86-64.so.2".
- On Windows, the loader is a logical component that encompasses all APIs in ntdll starting with "ldr".
- Its main role is to load shared libraries into the user address space and replace certain entries in their tables.

Slide 25 - C++ and Shared Libraries

In this final slide, Ofek Shilon discusses the lack of consideration by the C++ standard for shared libraries, stating that it makes the standard impractical in reality. He suggests that, in his opinion, clauses related to shared libraries should be removed from the standard to avoid impractical declarations. He also proposes revising the definitions of the notions of program and implementation.

Conclusion

During the conclusion of his conference, Ofek Shilon mentions that several topics, such as "relocation" or "versioning," were left unaddressed. He also expresses gratitude to Michael Kerrisk, who provided feedback on the initial draft of his presentation. He concludes his conference by answering questions from the audience, although this part is not included in the replay.